Linked List in C - Full Code Explanation

Linked List in C - Building and Displaying a List Using User Input

The Complete Code

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

int main() {
    struct node *start = NULL, *temp, *last = NULL;
    int n, val;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    // Build the linked list
    for (int i = 0; i < n; i++) {
        printf("Enter value %d: ", i + 1);
        scanf("%d", &val);

        temp = (struct node *)malloc(sizeof(struct node));
        temp->data = val;
        temp->next = NULL;

        if (start == NULL)
            start = temp;      // first node
        else
            last->next = temp; // attach to end

        last = temp;
    }

    // Display the full linked list
    temp = start;
    printf("Linked list: ");
    while (temp != NULL) {
        printf("%d->", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");

    return 0;
}
```

---

1. Header Files

```c
#include <stdio.h>
#include <stdlib.h>
```

- **`stdio.h`** gives access to `printf` and `scanf` (input/output functions).
- **`stdlib.h`** gives access to `malloc` (dynamic memory allocation) - needed because nodes are created at runtime, not at compile time.

---

2. The `struct node` Definition

```
        struct node {
            int data;
            struct node *next;
        };
```

This defines the **blueprint** for a linked list node. Every node has two parts:

- **`data`** - stores the actual value (an integer here).
- **`next`** - a pointer to another `struct node`. This is what links one node to the next, forming a chain.

This must say `struct node *next` (not `int node *next`) because a node needs to point to *another node of the same type*. This is called a **self-referential structure**.

The struct is placed *outside* `main()` (globally) so that any function in the program can use `struct node`, not just `main()`.

---

3. Variable Declarations

```
        struct node *start = NULL, *temp, *last = NULL;
        int n, val;
```

Three pointer variables of type `struct node*` are declared:

Variable	Purpose
`start`	Always points to the **first node** of the list - the "entry point" needed to walk the whole list later. Initialized to `NULL` since the list is empty at first.
`temp`	A **working/temporary** pointer, reused for two jobs: creating new nodes, and later traversing the list to print it.
`last`	Points to the **most recently added node**, so the next new node can be attached to it. Initialized to `NULL` since there is no "last node" yet.

`n` stores how many nodes the user wants to create; `val` temporarily stores each value the user types in.

---

4. Reading the Number of Nodes

```
        printf("Enter number of nodes: ");
        scanf("%d", &n);
```

Asks the user how many nodes to create and reads that number into `n`. `&n` passes the **address** of `n` so `scanf` can write the input value directly into that memory location.

---

5. The Building Loop

```
        for (int i = 0; i < n; i++) {
            printf("Enter value %d: ", i + 1);
            scanf("%d", &val);

            temp = (struct node *)malloc(sizeof(struct node));
            temp->data = val;
            temp->next = NULL;

            if (start == NULL)
                start = temp;
            else
                last->next = temp;

            last = temp;
        }
```

This loop runs `n` times, once per node.

Step 1 - Get input

```
        scanf("%d", &val);
```

Reads one integer from the user into `val`.

Step 2 - Create a new node

```
        temp = (struct node *)malloc(sizeof(struct node));
```

- `malloc(sizeof(struct node))` asks the operating system for a chunk of memory big enough to hold one `struct node` (space for an `int` plus a pointer), and returns the address of that memory.
- `malloc` returns a generic `void*`, so `(struct node *)` **casts** it to the correct pointer type so `->data` and `->next` can be used on it.
- `temp` now points to a brand-new, empty node somewhere in memory.

```
        temp->data = val;
        temp->next = NULL;
```

- Fills the new node's `data` field with the value just read.
- Sets `next` to `NULL` - this new node doesn't point anywhere yet. It will either become the last node, or get linked to a future node in a later iteration.

Step 3 - Attach it to the list

```
        if (start == NULL)
            start = temp;
        else
            last->next = temp;
```

This is the key logic:

- **First iteration**: `start` is still `NULL` (no nodes exist yet), so the new node *becomes* `start` - it is the first node of the list.
- **Every subsequent iteration**: `start` is no longer `NULL`, so instead the *previous* node (`last`) has its `next` set to point to this new node - physically linking them together.

Step 4 - Update `last`

```
        last = temp;
```

Whatever node was just created is now the newest "last node" in the list, so `last` is updated to point to it. This ensures the *next* iteration knows where to attach the following node.

Visual Trace (user enters 5, 10, 15)

Iteration	val	Action	List so far
i=0	5	`start = temp` (node with 5)	`start -> [5|NULL]`
i=1	10	`last->next = temp` (node(5)'s next = node(10))	`start -> [5] -> [10|NULL]`
i=2	15	`last->next = temp` (node(10)'s next = node(15))	`start -> [5] -> [10] -> [15|NULL]`

After the loop, `start` points to node `5`, which points to `10`, which points to `15`, which points to `NULL` - a complete linked list.

---

6. The Printing Loop

```
        temp = start;
        printf("Linked list: ");
        while (temp != NULL) {
            printf("%d->", temp->data);
            temp = temp->next;
        }
        printf("NULL\n");
```

`temp = start;` `temp` is reset to point to the **first** node again. It is being reused - earlier it was used to build the list, now it is repurposed to walk/traverse it. `temp` is used instead of moving `start` itself, because if `start` were changed, access to the beginning of the list would be permanently lost.

The `while` loop:

```
        while (temp != NULL) {
            printf("%d->", temp->data);   // print current node's value
            temp = temp->next;            // move to the next node
        }
```

- As long as `temp` points to a real node (not `NULL`), its `data` is printed, then `temp` advances to whatever `temp->next` points to.
- This continues until `temp` becomes `NULL`, which happens right after printing the **last** node (since the last node's `next` was set to `NULL` when it was created). So the loop correctly stops *after* printing everything, not before.

`printf("NULL\n");` After the loop ends, this prints `NULL` to visually mark the end of the list - the standard convention when displaying linked lists (e.g. `5->10->15->NULL`).

---

7. Why `malloc` and Pointers Are Necessary

Unlike an array (fixed size, decided at compile time), a linked list needs to grow dynamically based on user input - the value of `n` is not known in advance. `malloc` requests memory **at runtime**, one node at a

time, exactly when needed. Pointers (`next`) are what stitch these individually allocated, possibly scattered memory blocks into one logical sequence.

---

8. Sample Run

```
Enter number of nodes: 4
Enter value 1: 5
Enter value 2: 10
Enter value 3: 15
Enter value 4: 20
Linked list: 5->10->15->20->NULL
```

---

9. A Note on Good Practice

The program never calls `free()` on the allocated nodes before exiting. For a short program like this, the operating system reclaims that memory automatically when the program ends, so it will not cause visible problems here. However, in longer-running programs it is good practice to `free()` each node once it is no longer needed, to avoid memory leaks.